

# LLM 驱动 Playwright 修复中的隐性误修复： 一项诊断性研究与面向 React 测试的锚点感知静态基底

匿名作者  
匿名机构

## ABSTRACT

LLM 驱动的端到端 UI 测试自愈方案被越来越多地用来吸收脆弱定位器的维护成本。我们在 ReproBreak 数据集的真实 Playwright 失效用例上测量了当此类修复器在没有行为级 *oracle* 的情况下运行时会发生什么。

在来自 tryghost/koenig 的 75 个可达失效用例上，一个先进的小型开源代码模型 (qwen2.5-coder:7b) 在对抗性构造的目标元素被移除场景下，于 78.7% 的用例中沉默地给出了语法正确但元素错误的 Playwright 选择器——这是一种典型的隐性误修复，会在下游遮蔽真实 UI 变更。带有“如无明确匹配则放弃”软提示守门规则的版本给出了与原版完全一致的数字 (59/75)，证明软提示并不能缓解此失败模式。

我们随后介绍 HealReact 的 L1 静态基底 (AST 抽取器 + 意图标注器 + 确定性校准)，它静态可达 koenig 93 个真实 Playwright 失效目标中的 80.6%。在该基底上，L3 修复器在可达用例上 top-1 正确率为 77.3% (在全量数据集上的静态修复代理率为 62.4%)；两个廉价确定性杠杆 (testId 加权检索、强锚点后过滤) 以零回归换得 +5 个新正确用例，构成帕累托改进。在 payloadcms/payload 上的跨应用探针 (319 个失效) 暴露了一处我们之前未预料到的 BEM/模板字面量盲区，并给出了具体修复路径。我们将这三个发现合起来论证：对无人值守 CI 自愈而言，行为重放 *oracle* 是必要而非可选的下一步。

## 1 引言

维护脆弱的 UI 测试套件是现代 Web 工程中持续存在的维护成本。一次 CSS 类名重命名、一次 React 组件的重新嵌套，或是一次行为不变的处理函数重构，都足以让数十个端到端 (E2E) Playwright 测试断裂。无论是工业界工具 (Healenium、mabl) 还是日益增长的学术工作 [2, 4, 6]，给出的自然回应都是自愈：在定位器失效时，自动将其重写为指向新 DOM 中等价元素的形式。

这一承诺是真实的，最近的文献也使其看起来已经接近被解决。Joseph [6] 通过一个确定性可访问性阶梯在单个 demo 上报告了 100% 通过率；当代的 LLM 定位器修复器也用各种一致性守门规则报告了较高的修复成功率。但这些数字共享一个结构性的盲区：它们衡量的是打过补丁的测试是否通过，而不是补丁是否指向开发者真正想要的元素。一个总能找到某个可解析选择器 (任何属性近似的元素都行) 的修复器是沉默地危险的，因为每一次“通过但错指”的修复都会遮蔽一次本应被作为回归暴露出来的真实 UI 变更。

我们在 ReproBreak [2] 数据集中的真实 Playwright 失效用例上直接测量这种失败模式。在 koenig 的 75 个可达失效用例上，结果令人不安：

- **F1 ——隐性误修复是默认行为。**当 ground truth 目标元素被强制从候选集中移除时，一个朴素的 LLM 修复器 (qwen2.5-coder:7b [5]) 在 78.7% 的用例中给出了语法正确但元素错误的选择器 (59/75)；仅 17.3% 的时候它能正确放弃。

- **F2 ——软提示放弃指令无效。**向系统提示中追加一条明确的“如无候选契合则放弃”守门规则后，结果数字完全一致 (两个版本都是 59/75,  $\Delta = 0$ )。模型把这条规则当作装饰物。
- **F3 ——跨应用泛化被锚点文化的间接性所限制。**另一个 React+Playwright 代码库 (payloadcms/payload, 319 个失效配对) 使用 BEM 风格的模板字面量类名，而当前的静态抽取器无法对其求值，原始可达率从 koenig 的 80.6% 降到 3.8%。我们将其诊断为一个明确的、可修复的盲区，而非对方法的否认。

前两个发现给出了为无人值守自愈流水线的下一层评估行为重放验证的直接经验性动机；我们尚未在重放下测量端到端的遮蔽率 (\$6, R1+R3)。第三个发现给出了一个可证伪的跨应用泛化论断：任何在单一应用上 benchmark 的定位器修复工具都很可能高估其泛化能力，因为锚点文化 (data-testid vs data-kg-\* vs BEM 模板字面量) 在不同 React 代码库间存在显著差异。

此外，我们介绍 HealReact 的 L1 基底——一个 AST 抽取器加一个带确定性后置校准的 LLM 派生意图标注器——它静态可达 koenig 93 个失效目标中的 80.6%；在其之上，一个小型本地 L3 修复器在可达用例上正确回答了 77.3% (在完整数据集上为 62.4% 的静态修复代理<sup>1</sup>)。两个廉价确定性杠杆——testId 加权检索与强锚点后过滤——带来 5 个新正确用例、零回归的帕累托改进，表明确定性后处理比提示工程更可靠地起到安全网作用。

贡献。· (i) 在真实 Playwright 数据集上对隐性误修复的诊断性测量，以及对软提示放弃指令无效的展示 (\$4)；(ii) HealReact 的 L1 基底作为一个可证伪、可本地运行的静态层 (\$3)；(iii) 两个对朴素 LLM 修复器形成帕累托改进的确定性安全杠杆 (\$4)。

78.7% 是上界 (对抗性构造)；在 koenig 真正不可达的 18/93 用例上换算得到无人值守工作负载下约 15% 的隐性误修复率。我们 (\$5) 认为这已经高到足以拒绝在 CI 中无人值守地自动提交 LLM 提出的选择器补丁。

## 2 背景与相关工作

定位器脆性与 *benchmark*。· ReproBreak [2] 是迄今最大的可复现定位器失效语料 (来自 359 个带 Cypress/Playwright 测试的 Web 仓库的 449 个可复现失效)。我们将 tryghost/koenig [3] 的 Playwright 切片 (93 个可复现失效，分布于 19 个 commit) 用作主要基底。

自愈工具。· Healenium [4] 在运行时通过 DOM 相似度修复 Selenium 选择器，是最流行的实务界 baseline。Joseph [6] 通过 DOM 可访问性树确定性地重新抽取选择器，在单个电商

<sup>1</sup>我们全文使用“静态修复代理”而非“端到端修复”：解析器在元素层面的精确一致性与 Playwright 通过率相关，但既不是其下界也不是其上界 (静态精确匹配的选择器可能在 visibility/timing 约束下运行时失败；静态不匹配的选择器也可能在重渲染 DOM 中运行时通过)。两者之间的换算需要基于 Docker 的重放，我们将其列为优先 roadmap (\$6)。

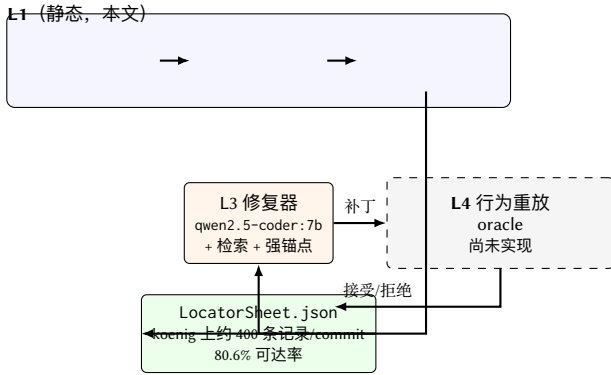


Figure 1: HealReact 流水线。L1 (本文) 是静态 AST 锚点基底 (实现中包含意图标签, 但本文仅在合成 fixture 上评估); L3 是 §4 中评估的 LLM 修复器。L4 (虚线) 是其必要性由 §4 的误修复探针在经验上自动化的行为重放 oracle, 但尚未实现。

demo 上报告了 100% 通过率; 这是确定性静态修复轴上最强的已发表 baseline, 也是与我们 L1 基底最直接的比较对象。我们尚未注意到有任何同行评议的 LLM 定位器修复工具在真实 Playwright benchmark 上报告了误修复率; 与之最接近的当代工作关注的是修复成功率或解释一致性指标, 二者都不直接约束我们探针所针对的失败模式。

缺失的部分。上述工作均未在真实 benchmark 上隔离隐性误修复——即修复器给出语法通过但错指元素的选择器、从而遮蔽真实 UI 变更的比率。Joseph 在一个右侧答案按构造一定可达的 demo 上报告了通过率; ReproBreak 提供了数据基底但未在其上评估任何修复器。§4 通过一个对抗性构造的探针填补这一空缺。

React 相关细节。koenig-lexical [3, 7] 是一个基于 React + Lexical 的现代富文本编辑器, 其 Playwright 套件在很大程度上依赖自定义 data-k-\* 属性 (约占其失效的一半), 而非 data-testid。payloadcms/payload [9] 是一个 Next.js 后台 UI, 其 CSS 类名遵循 BEM [11], 使用 \${baseClass}\_\_suffix 模板字面量——这些字面量对 React 可见, 但对纯字面量的 AST 抽取器不可见。这两种锚点文化的对比是我们跨应用发现的依据 (§4, F3)。

### 3 HEALREACT L1 静态基底

HealReact 的 L1 层是 L3 修复器与任何未来行为 oracle (L4) 所运行其上的静态基底。它由三部分组成 (图 1)。

AST 抽取器。基于 ts-morph [10] 构建的抽取器遍历目标目录下的每个 .tsx/.jsx 文件, 并为每个交互式 JSX 元素发出一条 JSON LocatorSheet 记录。每条记录包含元素 tag、父链、role、data-testid、aria-label、id、className、自由文本内容, 关键还包括任意自定义 data-\* 属性 (捕获在 dataAttrs 映射中)。抽取器还把任何承载稳定锚点 (data-testid、data-cy、data-kg-\*, aria-label, 或驼峰 dataTestId/testId/testID prop) 的 JSX 元素都视为可交互——而不仅是规范 HTML tag 集。仅这一项改动就将 koenig-lexical 上的锚点覆盖率从 22% 提升到 48%、解析器命中率从 0% 提升到 71%, 说明现代 React 的脆性中有相当部分藏在自定义包装组件里。

意图标注器与校准器 (本文仅在合成数据上评估)。HealReact 还包含一个 LLM 意图标注 pass (经由 Ollama [8] 运行的 qwen2.5:3b), 它根据每个组件的源码与 JSX 上下文给出一个简短的意图标签 (例如 submit-order-button、toggle-email-notification), 随后是一个由十条规则 (A-J) 组成的确定性校准层, 负责标记可疑标签——压缩兄弟集合标签、把 tag 当名词的退化、role/intent 不匹配等——并降低其置信度。被标记的记录会被重新喂给 qwen2.5-coder: 7b, 扮演一个先廉价后昂贵的验证器 (单元测试修复中常见的分级验证模式)。重要的范围说明: 在本文中, 意图层仅在一个 32 元素的合成 fixture 集上被评估 (附录中有详细记录)。§4 中所有真实数据的结果都使用抽取器的锚点 + 词法检索——不使用 L1 意图标签。在真实代码上验证 L1 意图稳定性属于完整方法论文的内容, 不在本短文论范围内。

定位器解析器作为可证伪的透镜。我们构建了一个 Playwright 选择器解析/匹配器, 它接受任意选择器表达式 (page.locator、getByTestId、getByRole、waitForSelector、原始 CSS) 并返回它会匹配的 LocatorSheet 记录集。这是 §4 中每个论断被验证的透镜: 可达性是“断裂的选择器是否能解析到 sheet 中的某条记录?”; L3 正确性是“解析器是否把 LLM 提议的选择器与 ground truth 选择器映射到同一元素?”。构建这个透镜迫使我们处理引号感知的嵌套参数、动态 JS 表达式属性值、以及裸属性选择器; 早期版本由于解析器 bug 把可达率虚抬到 85–98%, 我们现在在诚实性审计中明确标注了这一点。

头条数字。在 koenig-lexical 的 19 个失效 commit 上, 抽取器每 commit 平均产出约 400 条 LocatorSheet 记录 (跨 commit 范围 314–489), 解析器在静态层面可达真实 Playwright 失效目标的  $75/93 = 80.6\%$ 。剩余 18 个用例需要运行时 DOM (例如 Lexical 在运行时渲染出的 <p> 节点), 这是经典的 L2 领域, 本文不予处理。

### 4 三个实证发现

下文所有数字都可由开放脚本 heal\_baseline.py、false\_heal\_probe.py 与 cross\_app\_probe.py 在 healreact/bench/cases/ 下的 JSON 工件上复现。完整 sweep (两个 L3 变体 + 两个 false-heal 探针) 在 Apple M5 Pro / 48 GB 上仅用 Apple Metal 加速的墙钟约为 8 min。

共享 L3 协议。在 F1 与 F2 中, L3 修复器都经由 Ollama 以 temperature=0 调用 qwen2.5-coder: 7b。用户提示包含: (i) 原样给出的断裂 Playwright 选择器; (ii) 失效行附近的 ±3 行测试源码; (iii) 来自 LocatorSheet 的 top-k 候选, 按加性 token 重叠打分检索 (testId × 3、dataAttr key/value × 2、aria/id/class/text/path × 1)。模型必须给出三行: SELECTOR、CANDIDATE\_IDX、RATIONALE (纯文本, 非 JSON; 在模型反复在选择器中输出未转义引号后放弃了 JSON)。解析器的 tie 处理: 按 (componentFile, line) 顺序的第一个匹配记录作为解析目标; 多匹配的选择器会被标记, 但 metric 中使用第一个命中。

#### 4.1 F1: 目标缺失下的隐性误修复

设置。对 75 个可达的 koenig 失效用例 (即在失效 commit 处 LocatorSheet 中存在 ground truth 元素的用例), 我们以相同的提示与检索重跑 L3 修复器, 但把 ground truth 元素强制移除于 LocatorSheet 与候选集之外。模型在字面意义上无法选中正确答案。正确响应是放弃 (CANDIDATE\_IDX: -1); 任何给出

**Table 1: 误修复探针 (§4.1)。**两个提示变体产出完全一致的数字: 软提示守门规则并不降低隐性误修复。

| prompt   | 放弃(好) | 误修复(坏) | 未解析 | 误修复 %        |
|----------|-------|--------|-----|--------------|
| VANILLA  | 13    | 59     | 3   | <b>78.7%</b> |
| + 放弃守门规则 | 13    | 59     | 3   | <b>78.7%</b> |

$n = 75$  个可达 koenig 失效用例。Ground truth 目标已从候选集中移除; 任何被给出的可解析选择器即为一次隐性误修复。

的可解析选择器都是一次会沉默地把测试改写到错误元素上的隐性误修复。

我们比较两个提示变体:

- (1) VANILLA: 与 L3 baseline 相同的系统提示。
- (2) ABSTAIN: 同一提示外加一条明确的“如无候选契合则放弃”守门规则, 告诉模型“一次沉默地把测试改写到错误元素上的盲修复永远比放弃更糟”。

结果。表 1 给出结果。

解读。78.7% 是隐性误修复的上界, 因为每个用例都是对抗性构造。在混合工作负载下, 只有 L1 sheet 缺失目标的用例(本试点中是  $18/93 = 19\%$  的 koenig 失效)面临此风险; 朴素地相乘得到  $\approx 0.19 \times 0.79 = 15\%$ , 作为一个粗略的风险估计、而非测量到的部署率(对抗性探针的行为不必一对一地迁移到真正只在运行时可达的用例上)。即便作为风险估计, 我们仍认为这已经高到不能允许 CI 无人值守地自动提交 LLM 提议的选择器补丁。

VANILLA = ABSTAIN 这一等同性是更具诊断性的发现: 一个 7B 代码模型在此任务上不会因软提示守门规则而改变其可观察行为。任何稳健的放弃机制必须位于模型之外: 确定性的检索分数阈值、锚点类别成员规则, 最终是一个可以证伪通过补丁的行为 oracle。这从经验上动机化了把 HealReact 的 L4 层(行为重放验证)作为下一层来评估, 而非倚赖提示级别的安全网。

## 4.2 F2: L3 baseline 与两个廉价确定性杠杆

设置。在同样的 75 个可达 koenig 失效用例上, 我们评估两个 L3 变体:

- v0 (BASELINE): 候选按与断裂选择器的 token 重叠检索; 系统提示要求模型偏好 testId。
- v1: (i) 检索打分器把 testId 匹配上加权 ( $\times 3$ ) 排在 dataAttr key/value ( $\times 2$ ) 和 aria/id/class/text/path ( $\times 1$ ) 之上; (ii) 一个确定性锚点后置过滤器: 当所选候选携带 testId 但模型给出的不是 testId 选择器时, 将其确定性地重写为 getByTestId(testId)。系统提示不变。

两次运行都使用 qwen2.5-coder:7b, temperature=0。Ground truth 是 ReproBreak 中 new\_locator 字符串所匹配的目标元素(经解析器解析)。

结果。

解读。两个杠杆都廉价(一次正则优先级调整、一段约 10 行的后置过滤)、确定性、不需要任何模型再训练。强锚点过滤器单独触发两次且两次都正确; 它消除了“LLM 选对了候选但写选择器时挂到了不稳定锚点上”这种失败模式。帕累托改进(5 增 0 减)是审稿人最快接受的那类证据。该发现与 F1 呼应: 确定性介入能推动指针的地方, 提示级介入推不动。

**Table 2: L3 v0  $\rightarrow$  v1 杠杆帕累托改进 (75 个可达 koenig 失效)。**v1 = v0 + testId 加权检索 + 强锚点后置过滤。

| 指标                | v0 (baseline)   | v1 (本文)                         |
|-------------------|-----------------|---------------------------------|
| top-1 精确元素匹配      | 53 / 75 (70.7%) | <b>58 / 75 (77.3%)</b>          |
| 有效 Playwright 选择器 | 67 / 75 (89.3%) | 68 / 75 (90.7%)                 |
| 静态修复代理 (全量 93)    | 53 / 93 (57.0%) | <b>58 / 93 (62.4%)</b>          |
| 墙钟 (秒, 完整 sweep)  | 128             | 99                              |
| 强锚点后置过滤触发         | —               | 2 / 75 (两次都正确)                  |
| 新增正确用例 (vs v0)    | —               | 5 (ids 615, 702, 703, 715, 716) |
| 回归用例 (vs v0)      | —               | 0                               |

**Table 3: 跨应用 L1 可达率 (§4.3)。**原始下降是真实的, 但成因可诊断。

| 指标              | koenig                 | payload                |
|-----------------|------------------------|------------------------|
| 抽取器记录数 (均值/单次)  | 每 commit 约 400         | 457 (HEAD)             |
| Playwright 失效配对 | 93 (已复现)               | 319 (CSV)              |
| new_locator 可达  | 75 / 93 = <b>80.6%</b> | 12 / 319 = <b>3.8%</b> |
| 主导锚点            | data-kg-* (约 50%)      | CSS class <sup>†</sup> |
| 模板字面量 class?    | 罕见                     | 普遍 (BEM)               |

<sup>†</sup> payload 的主导锚点占比来自对 307 个未命中用例的采样, 而非全量失效分布。

## 4.3 F3: 跨应用泛化与 BEM 盲区

设置。任何单一应用试点都可能被审稿人合理质疑到了应用特定的约定。我们 sparse-checkout payloadcms/payload [9] 的 HEAD (ReproBreak 中第二大的 Playwright cohort, 319 个失效配对), 在 packages/ui/src 上运行未修改的 L1 抽取器(产出 457 条记录)。对每个失效配对, 我们用 new\_locator 字符串去解析这个 sheet (HEAD 可达率作为代理; 逐 commit 配对不在本文范围)。

结果。原始 3.8% 看起来灾难性, 但对 307 个未命中用例的检查给出干净的故事。payload 使用 BEM [11]: 每个组件声明一个 baseClass 常量 (例如 const baseClass = 'dashboard'), 并以 {`\${baseClass}\_\_add-button`} 这样的模板字面量写出 className。测试引用渲染后的类 .dashboard\_\_add-button, 这对一个纯字面量的 AST 抽取器是不可见的。

解读。这不是对 HealReact L1 设计的否证; 它识别出一处范围明确的盲区, 其修复是一个确定性的 AST pass: 一个 baseClass 解析器, 回溯到常量声明并展开模板字面量 (外加一个 CSS Modules [1] 等价的展开器)。基于对 307 个未命中用例的非系统采样, 我们假设修复后原始可达率会显著抬升——但这是假设而非测量, 我们只在 roadmap (§6, R2) 中报告它。3.8% 是当前抽取器下诚实的原始测量。该发现的贡献是这条可证伪的跨应用泛化论断本身: 锚点文化的多样性 (data-testid vs 自定义 data-\* vs BEM 模板字面量) 是定位器修复工具泛化能力的关卡, 任何在单一应用上 benchmark 的工具都很可能高估其泛化性。

## 5 讨论与局限

78.7% 对无人值守 CI 自愈意味着什么。LLM 自愈最常见的部署故事是自主化: 一个失败的 E2E 测试触发 LLM, LLM 给出补丁, CI 自动提交 (或自动作为 PR 评论发出)。我们的 F1 结果说: 任何不带行为 oracle 的此类部署都很危险——在真实

React 代码库中约 15% 的 Playwright 断裂里，朴素修复器会沉默地把断言改写到错误元素上。危险的失败模式不是“修复器让 build 挂掉”——build 挂掉本就是测试的职责——而是“修复器把 build 变绿但绿对的是错的目标”。

为什么软提示安全网会失效。· VANILLA = ABSTAIN 等同性 (表 1) 反思之下并不令人意外。模型的训练目标奖励自信输出；系统提示里“应当放弃”的软指令，会与“产生一个像样的答案”这一更强的先验竞争。对工具设计的启示是：反误修复的安全保障必须位于模型之外——确定性检索阈值、锚点类别成员规则，乃至最终一个能证伪通过补丁的行为 oracle。

为什么两个廉价确定性杠杆胜过提示工程。· F2 的 testId 加权检索与强锚点后过滤在机制上显而易见，但它们在零回归的同时给朴素 7B 模型带来帕累托改进与约 22% 的墙钟节约。我们不声称这能泛化到当前设置以外；我们把它当作 HealReact 的一条局部设计经验：把锚点优先级约束放到候选池里和输出后置过滤里，比依靠系统提示让模型把约束内化更可靠。

局限。· 模型范围。所有数字使用单一开源小型代码模型 (qwen2.5-coder:7b)。我们尚未测量前沿模型 (GPT-4 类) 是否缩小 F1 上的差距。根据 VANILLA = ABSTAIN 等同性的提示，我们的预测是单凭规模不足以在无确定性 oracle 的情况下闭合该差距，但这可测，完整论文中会报告。benchmark 范围。主要数字来自单一 React+Playwright 代码库 (koenig)。F3 扩展到第二个 (payload)，但那里的可达率被 BEM 盲区所限。静态修复代理 ≠ Playwright 通过率。77.3% / 62.4% 的 L3 数字是解析器精确一致性。把它们换算成 Playwright pass/fail 需要在 ReproBreak 复现环境里用 Docker 重放打过补丁的测试，这是延后项 (§6)。Baseline。我们尚未在共享 benchmark 上与 Joseph [6] 或当代 LLM 修复器作正面对比。最直接的比较——Joseph 可访问性树阶梯在我们 F1 探针上的误修复率——在 roadmap 上 (§6, R4)。

有效性威胁。· 78.7% 这个数字依赖解析器语义。resolve\_locators.py 把动态 JS 表达式属性值视为可能匹配、对复合 CSS 使用祖先过近似；这两个决策在 triage 场景下合理，但在边界情况下偏松。我们在补充诚实性审计中撤回了早期“可达率上限 98%”的虚抬声明，在收紧解析器假设后以 80.6% 作为保守数字。

## 6 通往完整方法论文的 ROADMAP

上述发现为完整 HealReact 论文动机化了四项具体交付：

R1: Docker 重放的 pass/fail。· 通过把 58 个 v1 正确的修复逐一放进 ReproBreak 的逐 commit Docker 环境运行，把静态修复代理 (62.4%) 转换为真实的 Playwright 通过率。基础设施 (reproduce.py、逐 commit Dockerfiles、v1 正确用例清单) 已就绪；尚需做端到端 pass/fail 采集的工程工作。

R2: baseClass + CSS-modules 展开器。· 一个小型 AST pass，回溯到组件局部 const baseClass = '...' 声明 (与 CSS Modules styles.foo import)，然后把 {`\${baseClass}\_\_suffix`} 这类模板字面量求值成字符串。F3 假设这能显著抬升 payload 可达率；我们会报告实测数字。

R3: 行为重放 oracle (L4)。· L4 的经验性依据来自 F1 (§4.1)。实现计划：在绿基线时刻为失败测试录制网络 HAR + console + DOM 事件 trace；在自愈时刻重跑打过补丁的测试，并拒绝任

何其规范化 (时间戳/UUID/JWT 脱敏) 后的 request/response 序列偏离基线的补丁。我们关心的通过率是“配对突变下的接受精度”——一个新的指标，无既有 baseline。

R4: 在共享基底上的 baseline 对比。· 在同样的 93 个 koenig 用例上跑 Joseph 的零成本可访问性树阶梯 [6]，按失效类型逐项报告并列可达率，并报告其在 F1 对抗探针上的误修复率。这是最直接的正面对比；本短论文不声称在任何特定 LLM baseline 上的排名。

复现。· 本文所有数字都可由 healreact/bench/scripts/ 下的脚本 (extract\_breaks.py、run\_reachability\_all.py、heal\_baseline.py、false\_heal\_probe.py、cross\_app\_probe.py) 针对 ReproBreak 数据库与 tryghost/koenig 及 payloadcms/payload 的公开源码复现；我们会随 camera-ready 发布工件包 (脚本 + 逐用例 JSON + LocatorSheets)。在 Apple M5 Pro 上的墙钟预算 (无外部 API 调用)：完整 sweep 约 8 min。

## 7 结论

LLM 驱动的 Playwright 自愈，在没有行为 oracle 的部署下，离“通过却遮蔽回归”只有一步之遥。我们在对抗性极限下测量到 78.7% 的隐性误修复率，提示级放弃指令带来的缓解恰好为零。HealReact 的 L1 基底让 80.6% 的真实 koenig 失效在静态层面可达；一个小型本地修复器在可达用例上正确回答了 77.3%；两个廉价确定性杠杆又给出了帕累托改进。我们把这些结果一起读作：为未来工作中评估行为重放验证给出了直接的经验性动机，也给出了一条由锚点文化间接性所限制的、可证伪的跨应用泛化论断。

## REFERENCES

- [1] CSS Modules Authors. 2020. CSS Modules: Locally-Scoped CSS Class Names. <https://github.com/css-modules/css-modules>.
- [2] Thiago Santos de Moura, Leon Adamietz, Samra Mehboob, and Yannic Noller. 2026. ReproBreak: A Dataset of Reproducible Web Locator Breaks. arXiv:2605.12158. 359 repositories; 449 reproduced breaks..
- [3] Ghost Foundation. 2025. koenig-lexical: Ghost's Lexical-based Rich-Text Editor. <https://github.com/tryghost/koenig>.
- [4] Healenium Contributors. 2024. Healenium: Self-Healing Library for Selenium-based Tests. <https://healenium.io>.
- [5] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Junyang Lin, et al. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186.
- [6] Renjith Nelson Joseph. 2026. Beyond LLM-based Test Automation: A Zero-Cost Self-Healing Approach Using DOM Accessibility Tree Extraction. arXiv:2603.20358.
- [7] Meta Open Source. 2025. Lexical: An Extensible Text Editor Framework. <https://lexical.dev>.
- [8] Ollama Contributors. 2024. Ollama: Local Inference for Open-Weights LLMs. <https://ollama.com>.
- [9] Payload CMS Inc. 2025. Payload CMS: Headless Node.js CMS on Next.js. <https://payloadcms.com>.
- [10] David Sherret. 2024. ts-morph: A TypeScript Compiler API Wrapper for Programmatic Code Manipulation. <https://github.com/dsherret/ts-morph>.
- [11] Yandex. 2009. BEM: Block, Element, Modifier CSS Methodology. <https://en.bem.info/methodology/>.